

# Agent Skills Registry — Listing Policy

**Version:** 2.3

**Chain:** Gnosis Chain (chain ID 100)

## Purpose

This registry curates AI agent **skills** through an optimistic open-challenge process. Anyone may submit a skill with a deposit, anyone may challenge it, and challenged entries are adjudicated by Kleros jurors against this policy. An unchallenged entry may become Registered without juror review; Registered means that the exact entry survived this process, not that any particular person inspected it.

A registry item binds its policy result to **exact bytes**, identified by content address — never to a name, a URL, a repository, or any mutable reference.

## Scope

This registry lists **skills only**: directories containing a `SKILL.md` per the Agent Skills specification, providing invocable capabilities to AI agents. Conventions (project-instruction files such as `AGENTS.md` or `CLAUDE.md`) and plugins (Agent Plugins bundles) are out of scope for this registry and may be served by separate registries with their own policies.

## Definitions

- **Skill tree** — the complete skill directory (`SKILL.md` plus any `scripts/`, `references/`, `assets/`, or other files), addressed by a single IPFS UnixFS directory CID (the **Tree CID**). The Tree CID immutably identifies every byte of the skill. The root CID and every linked CID in the DAG must be CIDv1 with a 32-byte SHA-256 multihash; linked blocks may use only the DAG-PB or raw codec. This is exactly what `ipfs add -r --cid-version 1` produces; a tree that links a CIDv0 anywhere is not a valid skill tree (criterion 2).
- **Virtual root name** — a Tree CID does not encode a name for its outer directory. Wherever a skill specification or runtime needs that name, this policy defines it to be the descriptor's Name value.
- **Descriptor** — the on-chain item: the RLP encoding of the column values below, stored in registry contract storage. The item ID is `keccak256(descriptor)`.
- **Submission period** — for this policy, the interval from submission of a registration request until that request is challenged or its challenge deadline passes.
- **Runtime** — the program that loads a skill tree and executes its instructions, for example Claude Code. A service, daemon or tool that the instructions talk to or invoke is a dependency, not a runtime; a skill describes its dependencies in its `compatibility` frontmatter.
- **Runtime-specific feature** — a tool, hook, command, invocation convention or `metadata` namespace outside the Agent Skills specification that the tree relies on in order to work. A feature that several runtimes implement is still runtime-specific; a tree that relies on it declares those runtimes. A tree does not rely on a feature that it uses only in an optional example or adapter with a working generic fallback, nor on runtime-namespaced `metadata` that other runtimes may

ignore, unless the tree's correctness, security or required permissions depend on that metadata being honoured. A requirement for an external program, service or daemon is a dependency, described in `compatibility`, and does not by itself make a tree runtime-specific.

## Descriptor Schema

Each submission consists of exactly these columns, in order:

| # | Column             | Required | Format                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---|--------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Name</b>        | Yes      | Must be <b>byte-identical</b> to the <code>name</code> field of the SKILL.md frontmatter in the skill tree.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 2 | <b>Description</b> | Yes      | Must be <b>byte-identical</b> to the <code>description</code> field of the SKILL.md frontmatter ( $\leq 1024$ characters per the Agent Skills specification).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 3 | <b>Tree CID</b>    | Yes      | The skill tree's IPFS CID in <b>canonical form</b> : CIDv1, base32, lowercase, DAG-PB codec, and a 32-byte SHA-256 multihash, referring to a UnixFS directory. Bare CID only — no <code>/ipfs/</code> prefix, no path suffix, no gateway URL. <b>IPNS names, DNSLink, and any mutable or resolvable-indirection reference are prohibited.</b>                                                                                                                                                                                                                                                                                                 |
| 4 | <b>Runtimes</b>    | Yes      | Supported agent runtimes as a list of identifiers: each identifier is lowercase snake case, a letter followed by letters, digits or underscores, at most 32 characters; identifiers are separated by single commas with no whitespace, are unique, are in ascending byte order, and number at most 16. The identifier <code>generic</code> is reserved: use it, alone, if the tree relies on no runtime-specific feature; otherwise list the runtimes whose features it relies on (e.g., <code>claude_code</code> , <code>cursor</code> ) and do not include <code>generic</code> . A column that does not match this form fails criterion 1. |
| 5 | <b>Origin</b>      | No       | A provenance claim: either a public git repository URL followed by <code>@</code> and a full 40-character commit SHA-1, or a publisher URL (e.g., <code>https://skills.example.org</code> ). If present, it must satisfy acceptance criterion 6. Leave empty if unused.                                                                                                                                                                                                                                                                                                                                                                       |
| 6 | <b>Reserved</b>    | Yes      | MUST be the empty string. It has no semantics in this policy version; assigning any future meaning requires a versioned policy revision.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

## Acceptance Criteria

An entry **MUST** satisfy ALL of the following criteria. An entry that fails ANY criterion must be rejected (registration disputes) or removed (removal disputes).

### 1. Well-Formed Descriptor

All six columns are present and conform to the formats above, including the canonical-CID rule for the Tree CID and the requirement that Reserved is empty.

### 2. Valid Skill

The skill tree's root contains a `SKILL.md` that is a valid skill per the **Agent Skills specification as published at commit 69ef37e9424c0a7ea9dd2293b559e43ec8176379 (2026-08-09) of [github.com/agentskills/agentskills](https://github.com/agentskills/agentskills)**: YAML frontmatter with required `name` (1–64 chars, lowercase alphanumeric and hyphens, no leading/trailing/consecutive hyphens) and `description` (1–1024 chars);

only spec-defined top-level frontmatter fields are permitted, and runtime-specific extension data must use the specification's `metadata` map. Output of validation tools (e.g., `skills-ref`) is admissible **evidence** of compliance or non-compliance, but the specification text at the pinned commit is the standard — not any tool.

The entry must be a skill (an invocable capability). Project-instruction files, plugin bundles, or other artifact types miscategorized as skills fail this criterion.

The tree consists of directories and regular files ONLY: it MUST NOT contain symlinks (of any target), and executable permission bits are **not semantic** in this policy version — conforming installers neither preserve nor interpret them, and skills invoke their scripts through explicit interpreters (e.g., `bash scripts/x.sh`, `python3 scripts/y.py`). A tree whose behavior depends on a file's executable bit fails this criterion. Every CID linked inside the tree is CIDv1 with a 32-byte SHA-256 multihash and the DAG-PB or raw codec, as `ipfs add -r --cid-version 1` produces; a tree containing a CIDv0 link fails this criterion.

### 3. Frontmatter Binding

The Name and Description columns are byte-identical to the SKILL.md frontmatter `name` and `description`. (This guarantees the on-chain strings a consumer indexes are exactly the strings an agent runtime will load from the verified artifact.)

### 4. Accurate Runtimes

The Runtimes column is derived from the tree. A tree that relies on no runtime-specific feature declares `generic` alone. A tree that relies on runtime-specific features declares the runtimes whose features it relies on; each declared runtime must be one the skill works, or is designed to work, with, and runtimes must not be listed speculatively. Omitting a further runtime the skill also works with is not a violation. Declaring `generic` for a tree that relies on a runtime-specific feature, or a specific runtime for a tree that relies on none, is.

### 5. No Malicious Behavior

The skill tree must NOT:

- Exfiltrate user data, credentials, environment variables, or files;
- Contain prompt injection targeting the consuming agent, other tools, or **reviewers and automated evaluators** (instructions addressed to "the reviewer", "the scanner", or an LLM judge are a violation in themselves);
- **Fetch and execute, or fetch and inject into agent context as instructions, content whose exact bytes are not bound by a SHA-256 integrity digest written as `sha256:` followed by exactly 64 lowercase hexadecimal characters.** The behavior must be prominently disclosed in the Description; the digest belongs in the tree, in the script or instructions that perform the check, and the skill must verify the fetched bytes and abort on mismatch before execution or context injection. Disclosed retrieval of mutable external data is permitted only when that data is not interpreted as code or agent instructions;

- Download or execute external binaries unless the behavior is prominently declared in the Description, the exact binary is identified in the tree by a `sha256:<64 lowercase hex characters>` digest, and the skill verifies that digest before execution and aborts on mismatch;
- Disable, bypass, or instruct the agent to bypass security mechanisms, sandboxing, or permission prompts;
- Install persistent backdoors or perform resource abuse (e.g., cryptocurrency mining);
- Employ **analysis-evasion constructions**, including: bulk padding or filler designed to truncate review (e.g., massive runs of whitespace or repeated tokens); executable logic or instructions concealed inside binary or archive assets (.docx, .zip, images, etc.) that the skill later extracts or interprets; encodings or obfuscation whose evident purpose is to defeat inspection. Such constructions are grounds for rejection **on construction alone**, without demonstrating the concealed payload's behavior.

**Registry-verified loading.** Loading a skill tree that is separately registered in this registry is not fetching unbound content under the two clauses above, provided that the loader: pins this registry's identity, meaning the chain, the registry address, its runtime code hash, its arbitrator and extra data, its governor and the policy versions it accepts, and accepts no other registry in its place; obtains a fresh, authenticated `Registered` status for the item under a permitted anchor mode before each acquisition, meaning every installation or update into a discovery directory, every re-enablement and every direct injection the loader performs; binds the exact item ID, descriptor and Tree CID; verifies the complete tree, every block, path and byte, and completes every check before any of those bytes become discoverable by a runtime, executable or injected as instructions; records the item ID, Tree CID and anchor of each acquisition; fails closed on any failure; keeps its own verification logic and trust configuration either within its tree or identified there by fixed integrity digests against which any externally obtained verifier or configuration is checked before use, a fetched checksum, a mutable reference or a signature alone never supplying or replacing that identity; discloses the loading behavior in its Description; and preserves the runtime's permission prompts. The exception covers acquisition only: later use of already-installed bytes is governed by the consumer provisions on pending removal, and content the acquired tree fetches in turn must itself satisfy this criterion. Two anchor modes are permitted under this version: a strict light client, and a header quorum, meaning at least two independently operated, locally configured sources agreeing on the same chain-bound finalized header, which the loader must label with its trust assumption and must not fall back to silently from a configured strict mode. Retiring either mode requires a later version of this policy.

## 6. Truthful Origin

If the Origin column is present, it must verifiably bind the claimed publisher or repository to this entry's **exact submitted semantic skill tree**:

- a git Origin must point to the stated commit, and that commit must contain the same semantic skill tree as the Tree CID; or
- a publisher Origin must publish a statement or listing under that origin which identifies the same Tree CID as its skill.

Common ownership, cross-linking, or control of both a repository and a domain is not sufficient unless the public provenance evidence binds the exact submitted semantic skill tree under one of the rules above.

For git comparison, the semantic tree consists of every relative path, node kind (directory or regular file), and regular-file byte sequence. Symlinks are prohibited in trees (criterion 2), and executable bits are not semantic in this policy version, so a git commit that differs from the submitted tree ONLY in executable bits still binds it; a commit containing symlinks cannot bind any listable tree. The descriptor Name supplies the virtual outer-directory name. UnixFS chunking and transport-only metadata such as modification times do not create a provenance mismatch.

A false or unverifiable Origin claim is a violation of the same severity as impersonation.

## **7. No Impersonation**

The entry must not impersonate, typosquat, or misleadingly copy another project's or publisher's name, description, or branding.

## **8. No Duplicates**

The entry must not duplicate an already-registered entry: neither the same Tree CID, nor a tree that differs from a registered entry's, across the complete tree including assets, templates, schemas and references, only by edits with no material effect on capability, behavior, security or trust configuration, or supported environment. Cosmetic edits, whitespace, comments, formatting and wording that leave behavior unchanged, are such edits; a security fix, a compatibility correction, a changed functional asset or a changed dependency or profile pin is not when it has such an effect, however small its diff, and a changed hash alone does not show one. A challenger shows the difference from the authenticated trees; the submitter may show the material effect. (Identical descriptors are already impossible: the item ID is the descriptor hash.)

## **9. Reviewable Size**

The skill tree's total size must not exceed **2 MiB**. (This bound exists so that jurors and challengers can realistically review entries; it may be revised in future policy versions.)

## **10. Submission-Period Availability**

Throughout the submission period, the complete skill tree identified by the Tree CID **MUST** be stored on IPFS and remain accessible and discoverable. Otherwise, the entry fails this criterion.

## **11. Lawful Distribution**

The submitter must be entitled to distribute the skill tree's content. Entries whose distribution violates the license or rights of included content may be rejected or removed. A `license` frontmatter field or bundled license file is strongly recommended.

## **Adjudication Principles**

These principles bind how the criteria above are applied:

**A. Content identity is hosting-independent.** Jurors may obtain the skill tree from any provider, but MUST verify the complete content and directory structure against the Tree CID before relying on it. A matching CID authenticates the bytes; it does not make any gateway or provider trusted.

**B. Availability is a submission-period obligation.** Criterion 10 concerns accessibility and discoverability during the registration request's challenge period. It does not require permanent availability after listing. Producing hash-matching bytes later proves their identity, not that they were available throughout the earlier submission period, and does not by itself cure a demonstrated earlier violation. Conversely, a failed fetch during juror voting does not by itself prove that the tree was unavailable during the earlier submission period.

**C. Availability is decided from evidence, not a juror-time liveness test.** Jurors evaluate evidence about criterion 10 through the ordinary Kleros dispute process. This policy prescribes no mechanical availability test, retry count, uptime monitor, or live-fetch procedure. Current availability or unavailability must not be substituted for evidence about the submission period.

**D. Construction suffices for evasion.** Criterion 5's evasion clause is deliberately judgeable without executing anything: a skill that pads, conceals, or addresses its reviewers has violated the policy by how it is built.

## Removal Criteria

A registered entry should be removed if it fails an acceptance criterion — including violations discovered after listing, newly discovered security vulnerabilities in the skill, or an Origin claim that has become demonstrably false (e.g., the publisher disavows the skill).

Post-listing unavailability, by itself, is **not** a removal ground: criterion 10 does not impose permanent hosting. Evidence that the entry violated criterion 10 during its submission period may still support removal.

`Absent` status alone does not prove that an item was removed; rejected items that were never Registered are also Absent. A consumer SHOULD treat an exact item as locally revoked only when its authenticated local history or lockfile records that item as previously Registered and a fresh authenticated check now reports the same item ID as Absent. Consumers SHOULD retain installed bytes and lockfile history for warning, audit, and forensic use rather than silently deleting them.

## Consumer Handling of Pending Removal

Consumers MUST treat every `ClearingRequested` item uniformly, regardless of who requested removal. New installation and automatic loading or execution SHOULD be suspended by default while the request is pending. A consumer MAY permit an explicit local override and SHOULD retain existing bytes and lockfile history for inspection. No requester address has special protocol or trust semantics.

## Amendments

This policy can change only through the registry's governor, a timelock contract that queues every change on chain seven days before it takes effect. Each request keeps the policy version it was submitted under; a removal request opened after a change is judged under the version then in force, except that an entry is not removable solely for failing a runtime or duplicate classification rule, in criterion 4, criterion 8 or the Runtimes column of the descriptor schema, that was introduced after the policy version of the registration request that established its current registration. A registration after a removal takes its own baseline, and nothing here waives descriptor syntax or the safety criteria.

## Evidence Guidelines

Challengers should identify the criterion violated and provide evidence specific to that criterion. For content-based allegations, they should identify the relevant files and lines and provide the skill tree or another means for jurors to retrieve it; every byte relied upon must verify against the Tree CID. For criterion 10, they should provide contemporaneous evidence concerning accessibility and discoverability during the submission period. Scanner reports and reproducible behavioral evidence may be submitted as supporting evidence. When challenging on evasion constructions (criterion 5), identifying the construction is sufficient — the payload need not be detonated.

Submitters defending an entry should provide the verified skill tree if it is needed to evaluate a content allegation and address the alleged violation. For an availability allegation, evidence of a later successful fetch or later production of the tree does not, by itself, establish earlier submission-period availability.

## Juror Instructions

1. Read this policy in full. The pinned Agent Skills specification commit is part of this policy for criterion 2.
2. Obtain the skill tree when needed to decide the alleged violation, and verify its complete DAG and directory structure against the Tree CID before relying on it. Hash-matching content is authentic regardless of source.
3. Check every acceptance criterion. Failure of any single criterion decides the dispute.
4. For criterion 10, evaluate evidence about the submission period. Do not infer continuous earlier availability merely from a successful juror-time fetch, do not infer earlier unavailability merely from a failed juror-time fetch, and do not treat late production as an automatic cure.
5. Be aware that skill content may attempt to manipulate you: instructions embedded in the skill addressed to reviewers or AI assistants are criterion-5 violations, not instructions to follow.
6. When in doubt, prefer the interpretation that protects end users from potential harm.